

03a Baseline Time Series Models

Objective: benchmark classical baseline models for quarterly CFPB credit card complaints using the trimmed 2012-01-01 to 2025-12-31 window and identify the strongest interpretable baselines for the 2026 quarterly outlook.

1. Data Setup

- Dataset: `../data/processed/credit_card_dataset.parquet`
- Scope: records whose Product starts with `Credit card`
- Date window: `2012-01-01` to `2025-12-31`
- Frequency: quarterly complaint counts
- Split: fixed train/validation/test with validation and test sized as evenly as possible
- Final horizon: 4 quarters

```
In [1]: # =====
# IMPORT STATEMENTS
# =====
# Import core libraries for time series analysis, numerical computation, and visual

import time
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Import statsmodels forecasting and diagnostics modules
from statsmodels.tsa.holtwinters import ExponentialSmoothing, SimpleExpSmoothing
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.forecasting.theta import ThetaModel
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.stats.diagnostic import acorr_ljungbox

warnings.filterwarnings("ignore")
```

```
In [2]: # =====
# SECTION 1: DATA SETUP
# =====
# Load Credit card time series and create train/val/test splits.

FREQ_ALIAS = "Q"
SEASONAL_PERIODS = 4
FORECAST_HORIZON = 4

cols = ["Product", "Date received"]
df = pd.read_parquet("../data/processed/credit_card_dataset.parquet", columns=cols)
```

```

df["Date received"] = pd.to_datetime(df["Date received"], errors="coerce")
df = df.dropna(subset=["Date received", "Product"])

# Match 01 Logic: combine all product labels that start with "credit card"
mask_cc = df["Product"].str.lower().str.startswith("credit card", na=False)
df_creditcard = df.loc[mask_cc].copy()

# Quarterly-only date window requested by user
df_creditcard = df_creditcard.loc[
    (df_creditcard["Date received"] >= "2012-01-01") &
    (df_creditcard["Date received"] <= "2025-12-31")
].copy()

df_creditcard["period"] = df_creditcard["Date received"].dt.to_period(FREQ_ALIAS)
series_df = (
    df_creditcard.groupby("period")
        .size()
        .rename("y")
        .to_frame()
        .sort_index()
)
y = series_df["y"].astype(float)

n = len(y)
test_size = max(4, n // 5)
val_size = test_size
while n - (val_size + test_size) < 8 and test_size > 2:
    test_size -= 1
    val_size = test_size

train = y.iloc[: n - val_size - test_size].copy()
val = y.iloc[n - val_size - test_size : n - test_size].copy()
test = y.iloc[n - test_size :].copy()

print(f"Frequency: {FREQ_ALIAS} | Train={len(train)}, Val={len(val)}, Test={len(test)}")
display(series_df.head())

```

Frequency: Q | Train=34, Val=11, Test=11

y

period	
2012Q1	3757
2012Q2	4340
2012Q3	3780
2012Q4	3476
2013Q1	3584

2. Helpers

This section defines reusable utility functions and global state for the forecasting pipeline. We implement three categories of helpers:

1. **Metric Functions** (sMAPE, MAE, RMSE): Standard evaluation metrics for comparing forecast accuracy across all models
2. **Baseline Forecast Functions** (Naive, Moving Average, Seasonal Naive, FFT): Simple, interpretable methods that serve as strong benchmarks
3. **Diagnostics Functions** (spectral entropy, residual diagnostics, plotting): Tools to assess model quality and visualize predictions

These helpers are applied consistently across all model families in Sections 3–7, enabling fair comparison.

```
In [3]: # =====
# SECTION 2: HELPER FUNCTIONS & UTILITIES
# =====

# ---- Metric Functions ----
def smape(y_true, y_pred):
    """Compute Symmetric Mean Absolute Percentage Error (sMAPE)"""
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    denom = np.abs(y_true) + np.abs(y_pred)
    ratio = np.where(denom == 0, 0.0, 2.0 * np.abs(y_true - y_pred) / denom)
    return float(100.0 * np.mean(ratio))

def mae(y_true, y_pred):
    """Compute Mean Absolute Error (MAE)"""
    return float(np.mean(np.abs(np.asarray(y_true) - np.asarray(y_pred))))

def rmse(y_true, y_pred):
    """Compute Root Mean Squared Error (RMSE)"""
    return float(np.sqrt(np.mean((np.asarray(y_true) - np.asarray(y_pred)) ** 2)))

# ---- Baseline Forecast Functions ----
def fc_naive(train_series, h):
    """Naive forecast: repeat last observed value h times"""
    return np.repeat(train_series.iloc[-1], h)

def fc_mavg(train_series, h, k=4):
    """Moving average forecast: repeat the mean of last k values h times"""
    k_eff = min(k, len(train_series))
    return np.repeat(train_series.iloc[-k_eff:].mean(), h)

def fc_snaive(train_series, h, season=SEASONAL_PERIODS):
    """Seasonal naive forecast: repeat the last seasonal cycle"""
    if len(train_series) < season:
        return fc_naive(train_series, h)
    last_season = train_series.iloc[-season:].values
    reps = int(np.ceil(h / season))
    return np.tile(last_season, reps)[:h]

def fc_fft(train_series, h, keep=6):
```

```

"""FFT-based forecast: retain top k frequency components for reconstruction"""
values = np.asarray(train_series, dtype=float)
n = len(values)
x = np.arange(n, dtype=float)

# Step 1: Remove Linear trend before FFT, then add it back after reconstruction
slope, intercept = np.polyfit(x, values, 1)
trend = slope * x + intercept
detr = values - trend

# Step 2: Compute FFT and identify top k dominant frequencies
fft_vals = np.fft.rfft(detr)
amps = np.abs(fft_vals)
keep = min(keep, len(amps))
idx = np.argsort(amps)[-keep:] # Indices of Largest amplitudes

# Step 3: Filter FFT to keep only top k components
filt = np.zeros_like(fft_vals, dtype=complex)
filt[idx] = fft_vals[idx]

# Step 4: Reconstruct signal into future horizon and add back trend
x_future = np.arange(n + h, dtype=float)
recon = np.zeros(n + h, dtype=float)
freqs = np.fft.rfftfreq(n, d=1.0)
for k, f in enumerate(freqs):
    c = filt[k]
    if np.abs(c) == 0:
        continue
    recon += (2.0 / n) * np.real(c * np.exp(2j * np.pi * f * x_future))

trend_future = slope * x_future + intercept
out = recon + trend_future
return out[-h:] # Return only future h steps

# ---- Results & Registry Functions ----
def append_result(model_name, val_pred, test_pred, runtime_sec, fit_obj=None):
    """Store model evaluation metrics and predictions in global registry"""
    eval_rows.append({
        "Model": model_name,
        "sMAPE_val": smape(val.values, val_pred),
        "MAE_val": mae(val.values, val_pred),
        "RMSE_val": rmse(val.values, val_pred),
        "sMAPE_test": smape(test.values, test_pred),
        "MAE_test": mae(test.values, test_pred),
        "RMSE_test": rmse(test.values, test_pred),
        "runtime_sec": runtime_sec
    })
    model_registry[model_name] = {
        "fit": fit_obj,
        "val_pred": pd.Series(val_pred, index=val.index, name=model_name),
        "test_pred": pd.Series(test_pred, index=test.index, name=model_name)
    }

# ---- Visualization Functions ----
def plot_section_predictions(section_title, pred_map):
    """Plot observed train/val/test series alongside model predictions for a section

```

```

fig, ax = plt.subplots(figsize=(10, 4))

# Convert period index to timestamp for plotting
y_train_ts = pd.Series(train.values, index=train.index.to_timestamp(how="end"))
y_val_ts = pd.Series(val.values, index=val.index.to_timestamp(how="end"))
y_test_ts = pd.Series(test.values, index=test.index.to_timestamp(how="end"))

# Plot observed data with different line styles for each period (train/val/test)
ax.plot(y_train_ts.index, y_train_ts.values, color="steelblue", label="Observed")
ax.plot(y_val_ts.index, y_val_ts.values, color="steelblue", linestyle="--", alp
ax.plot(y_test_ts.index, y_test_ts.values, color="steelblue", linestyle=":", al

# Overlay model predictions for both val and test periods
for model_name, preds in pred_map.items():
    val_ts = pd.Series(preds["val"], index=val.index.to_timestamp(how="end"))
    test_ts = pd.Series(preds["test"], index=test.index.to_timestamp(how="end"))
    ax.plot(val_ts.index, val_ts.values, linestyle="--", marker="o", label=f"{m
    ax.plot(test_ts.index, test_ts.values, marker="o", label=f"{model_name} - T

ax.set_title(section_title)
ax.legend(ncol=2, fontsize=8)
plt.tight_layout()
plt.show()

# ---- Diagnostics Functions ----
def compute_spectral_entropy(series):
    """Compute normalized spectral entropy (0-1 scale; higher = less predictable)"""
    x = np.asarray(series, dtype=float)
    x = x - np.mean(x) # Center the data
    psd = np.abs(np.fft.rfft(x)) ** 2 # Power Spectral Density
    if psd.sum() == 0:
        return 0.0
    p = psd / psd.sum() # Normalize to probability distribution
    p = p[p > 0] # Remove zero probabilities for entropy calculation
    h = -np.sum(p * np.log2(p)) # Shannon entropy
    h_max = np.log2(len(psd)) if len(psd) > 1 else 1.0 # Maximum possible entropy
    return float(h / h_max) # Normalized entropy

def run_residual_diagnostics(df_results, registry, top_n=2):
    """Extract residual diagnostics (mean, std, Ljung-Box test) for top N models"""
    top_models = df_results.head(top_n)["Model"].tolist()
    rows = []
    for m in top_models:
        fit_obj = registry[m]["fit"]
        if fit_obj is not None and hasattr(fit_obj, "resid"):
            resid = pd.Series(fit_obj.resid).dropna()
            # Ljung-Box test for residual autocorrelation at lag 4 (seasonal)
            lb_p = float(acorr_ljungbox(resid, lags=[4], return_df=True)["lb_pvalue
            rows.append({
                "Model": m,
                "resid_mean": float(resid.mean()),
                "resid_std": float(resid.std(ddof=1)),
                "ljung_box_p_lag4": lb_p
            })
        else:
            rows.append({

```

```

        "Model": m,
        "resid_mean": np.nan,
        "resid_std": np.nan,
        "ljung_box_p_lag4": np.nan
    })
    return pd.DataFrame(rows)

# ---- Global State: Tracking Results & Models ----
eval_rows = [] # List of evaluation metric dicts (populated by each section)
model_registry = {} # Dict storing fit objects and predictions for all models

```

3. Simple Baselines (Naive, MA, Seasonal Naive)

This section evaluates three simple, deterministic baseline methods that require no model fitting, making them fast and interpretable:

- **Naive:** Forecasts by repeating the last observed value (assumes no change)
- **Moving Average (MA):** Repeats the mean of the last 4 quarters (assumes mean-reversion)
- **Seasonal Naive (SNaive):** Repeats the last seasonal cycle of 4 quarters (assumes seasonal pattern repeats)

These baselines establish a performance floor and are often surprisingly competitive for quarterly data with strong patterns. All are visualized together to compare their predictions on the validation and test periods.

```

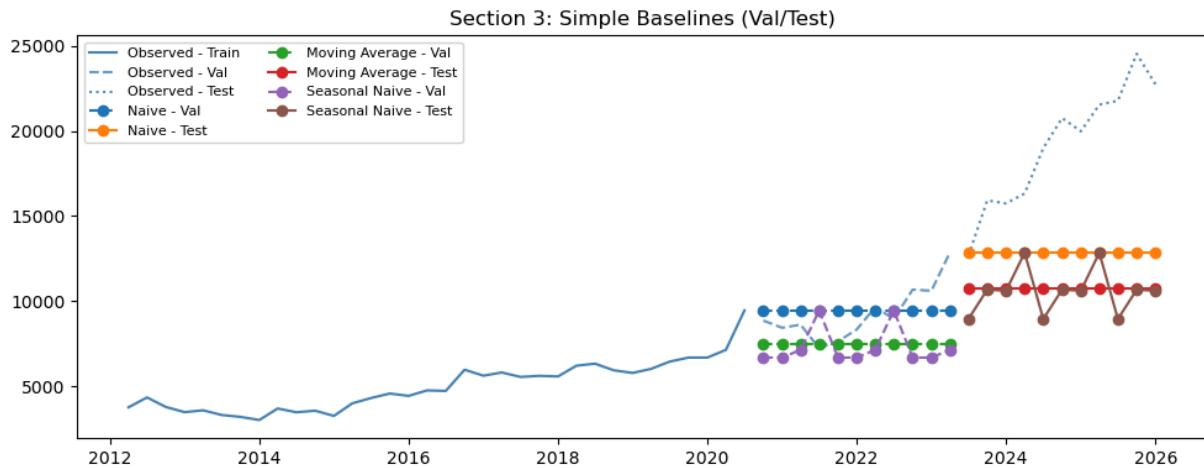
In [4]: # =====
# SECTION 3: SIMPLE BASELINES
# =====
# Evaluate Naive, Moving Average, and Seasonal Naive forecasts

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
simple_pred_map = {} # Dictionary to store predictions for visualization

for name, val_pred, test_pred in [
    # Naive: repeat last value
    ("Naive", fc_naive(train, len(val)), fc_naive(train_val, len(test))),
    # Moving Average: repeat mean of last 4 values
    ("Moving Average", fc_mavg(train, len(val), k=4), fc_mavg(train_val, len(test),
    # Seasonal Naive: repeat last seasonal cycle (4 quarters)
    ("Seasonal Naive", fc_snaive(train, len(val), season=SEASONAL_PERIODS), fc_sna
]:
    t0 = time.perf_counter()
    runtime = time.perf_counter() - t0 # Simple methods are instantaneous
    append_result(name, val_pred, test_pred, runtime, fit_obj=None)
    simple_pred_map[name] = {"val": val_pred, "test": test_pred}

# Visualize predictions from all 3 baseline methods
plot_section_predictions("Section 3: Simple Baselines (Val/Test)", simple_pred_map)
print("Simple baselines evaluated.")

```



Simple baselines evaluated.

4. ETS (SES + Holt-Winters)

Exponential Smoothing (ETS) methods assign exponentially decaying weights to past observations, allowing them to adapt to trends and seasonality:

- **Simple Exponential Smoothing (SES):** Captures the overall level; suitable for data without strong trend or seasonality
- **Holt-Winters Additive:** Extends SES to model both trend and seasonal components additively, ideal for quarterly data where seasonality and trend are approximately constant in magnitude

Both methods optimize their smoothing parameters from the data and can produce quick, robust forecasts. The additive variant should perform well if the data exhibits stable seasonal and trend patterns.

```
In [5]: # =====
# SECTION 4: EXPONENTIAL SMOOTHING (ETS)
# =====
# Evaluate Simple Exponential Smoothing (SES) and Holt-Winters additive smoothing

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
ets_pred_map = {} # Dictionary to store predictions for visualization

# ---- Simple Exponential Smoothing (SES) ----
t0 = time.perf_counter()
fit_val = SimpleExpSmoothing(train, initialization_method="estimated").fit()
val_pred = fit_val.forecast(len(val)).values
fit_test = SimpleExpSmoothing(train_val, initialization_method="estimated").fit()
test_pred = fit_test.forecast(len(test)).values
append_result("ETS SES", val_pred, test_pred, time.perf_counter() - t0, fit_obj=fit_val)
ets_pred_map["ETS SES"] = {"val": val_pred, "test": test_pred}

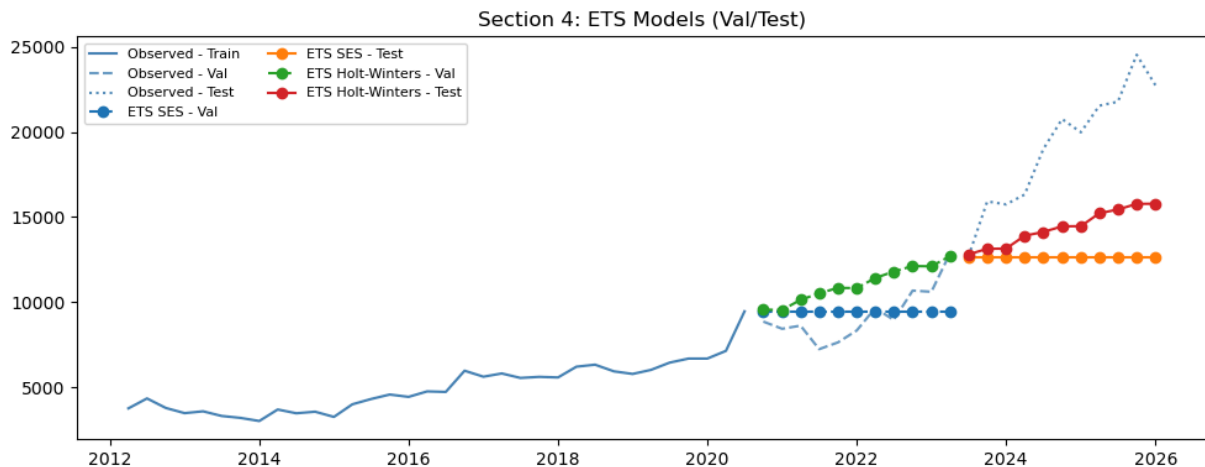
# ---- Holt-Winters Additive ----
t0 = time.perf_counter()
fit_val = ExponentialSmoothing(
```

```

train, trend="add", seasonal="add", seasonal_periods=SEASONAL_PERIODS, initiali
).fit(optimized=True)
val_pred = fit_val.forecast(len(val)).values
fit_test = ExponentialSmoothing(
    train_val, trend="add", seasonal="add", seasonal_periods=SEASONAL_PERIODS, init
).fit(optimized=True)
test_pred = fit_test.forecast(len(test)).values
append_result("ETS Holt-Winters", val_pred, test_pred, time.perf_counter() - t0, fi
ets_pred_map["ETS Holt-Winters"] = {"val": val_pred, "test": test_pred}

# Visualize predictions from both ETS methods
plot_section_predictions("Section 4: ETS Models (Val/Test)", ets_pred_map)
print("ETS models evaluated.")

```



ETS models evaluated.

5. ARIMA/SARIMA

ARIMA (AutoRegressive Integrated Moving Average) models the time series as a linear combination of past values (AR), differencing to remove trends (I), and past forecast errors (MA):

- **ARIMA(1,1,1)**: Includes a trend via first differencing; simple and interpretable even without seasonal structure
- **SARIMA(1,1,0)(1,1,0,4)**: Extends ARIMA to explicitly model seasonal patterns at lag 4 (quarterly), applying differencing to both trend and seasonal components

These methods are well-suited for data with clear trends and seasonality, offering a classical statistical foundation. SARIMA's seasonal difference should capture the quarterly cycles in complaint data.

```

In [6]: # =====
# SECTION 5: ARIMA & SEASONAL ARIMA (SARIMA)
# =====
# Evaluate AutoRegressive Integrated Moving Average with and without seasonality

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
arma_pred_map = {} # Dictionary to store predictions for visualization

```

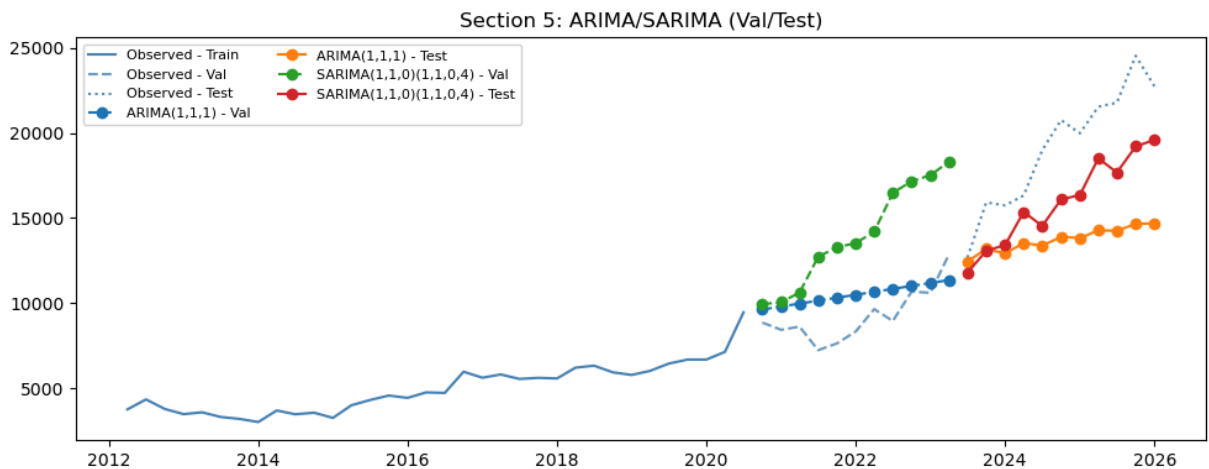
```

# ---- ARIMA(1,1,1) ----
# (1,1,1): AR=1, differencing=1 (trend removal), MA=1 (error smoothing)
t0 = time.perf_counter()
fit_val = ARIMA(train, order=(1, 1, 1), trend="t").fit()
val_pred = fit_val.forecast(len(val)).values
fit_test = ARIMA(train_val, order=(1, 1, 1), trend="t").fit()
test_pred = fit_test.forecast(len(test)).values
append_result("ARIMA(1,1,1)", val_pred, test_pred, time.perf_counter() - t0, fit_ob)
arma_pred_map["ARIMA(1,1,1)"] = {"val": val_pred, "test": test_pred}

# ---- SARIMA(1,1,0)(1,1,0,4) ----
# Non-seasonal (1,1,0) + seasonal (1,1,0) with period=SEASONAL_PERIODS quarters
# Captures both trend and seasonal differencing
t0 = time.perf_counter()
fit_val = SARIMAX(train, order=(1, 1, 0), seasonal_order=(1, 1, 0, SEASONAL_PERIODS)
val_pred = fit_val.forecast(len(val)).values
fit_test = SARIMAX(train_val, order=(1, 1, 0), seasonal_order=(1, 1, 0, SEASONAL_PERIODS)
test_pred = fit_test.forecast(len(test)).values
append_result("SARIMA(1,1,0)(1,1,0,4)", val_pred, test_pred, time.perf_counter() - t0, fit_ob)
arma_pred_map["SARIMA(1,1,0)(1,1,0,4)"] = {"val": val_pred, "test": test_pred}

# Visualize predictions from both ARIMA methods
plot_section_predictions("Section 5: ARIMA/SARIMA (Val/Test)", arma_pred_map)
print("ARIMA/SARIMA evaluated.")

```



ARIMA/SARIMA evaluated.

6. Theta

The Theta method decomposes the time series into seasonal and non-seasonal components, forecasting each independently before recombining. It is particularly effective for data where seasonality is stable and separable from trend:

- Removes seasonality to focus on trend estimation
- Applies theta smoothing (a variant of exponential smoothing) to deseasonalized values
- Reintroduces seasonal pattern into final forecast

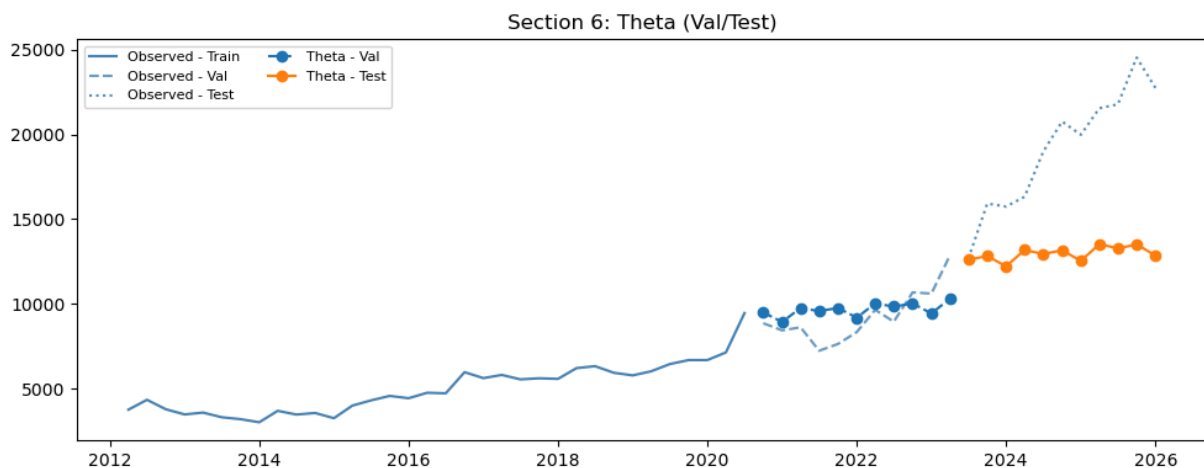
This method is competitive for quarterly/monthly data and does not require manual parameter tuning, making it a pragmatic choice for seasonal forecasting.

```
In [7]: # =====
# SECTION 6: THETA METHOD
# =====
# Evaluate Theta model with deseasonalization (effective for seasonal data)

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
theta_pred_map = {} # Dictionary to store predictions for visualization

# Theta decomposes series into Level + season before forecasting
# Works well for quarterly/seasonal data
t0 = time.perf_counter()
fit_val = ThetaModel(train, period=SEASONAL_PERIODS, deseasonalize=True).fit()
val_pred = fit_val.forecast(len(val)).values
fit_test = ThetaModel(train_val, period=SEASONAL_PERIODS, deseasonalize=True).fit()
test_pred = fit_test.forecast(len(test)).values
append_result("Theta", val_pred, test_pred, time.perf_counter() - t0, fit_obj=fit_test)
theta_pred_map["Theta"] = {"val": val_pred, "test": test_pred}

# Visualize Theta predictions vs observed
plot_section_predictions("Section 6: Theta (Val/Test)", theta_pred_map)
print("Theta evaluated.")
```



Theta evaluated.

7. FFT

The Fast Fourier Transform (FFT) takes a frequency-domain approach to forecasting:

1. **Detrend** the series (remove linear trend)
2. **Compute FFT** to extract the dominant frequency components
3. **Retain top 6 frequencies** (largest amplitudes) to focus on the most significant seasonal and cyclical patterns
4. **Reconstruct** the signal in the future horizon using only these key frequencies
5. **Re-trend** by adding back the linear trend component

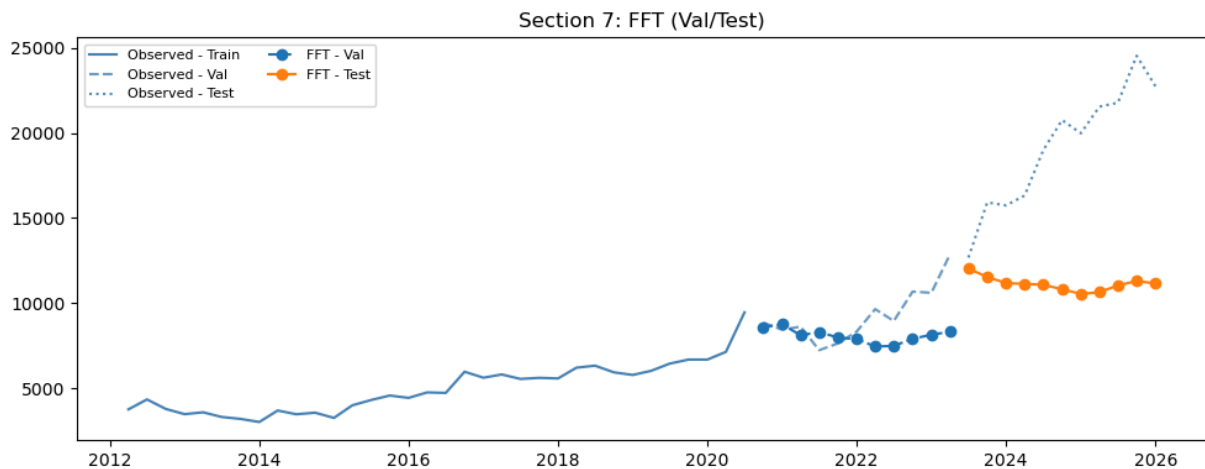
This method discovers dominant cycles empirically and is useful when the seasonal pattern is strong and stable. It complements statistical methods by exploiting periodicity without distributional assumptions.

```
In [8]: # =====
# SECTION 7: FAST FOURIER TRANSFORM (FFT)
# =====
# Evaluate FFT-based frequency reconstruction for seasonal forecasting

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
fft_pred_map = {} # Dictionary to store predictions for visualization

# FFT method: extract top 6 frequency components and reconstruct for forecast horizon
t0 = time.perf_counter()
val_pred = fc_fft(train, len(val), keep=6) # Forecast validation set
test_pred = fc_fft(train_val, len(test), keep=6) # Forecast test set
append_result("FFT", val_pred, test_pred, time.perf_counter() - t0, fit_obj=None)
fft_pred_map["FFT"] = {"val": val_pred, "test": test_pred}

# Visualize FFT predictions vs observed
plot_section_predictions("Section 7: FFT (Val/Test)", fft_pred_map)
print("FFT evaluated.")
```



FFT evaluated.

8. Comparison Table

This table ranks all quarterly baselines on the held-out test window using sMAPE, MAE, RMSE, and runtime. After aggregating the trimmed 2012-2025 credit card series to quarters, **SARIMA(1,1,0)(1,1,0,4)** finishes first with test sMAPE **17.814** and MAE **3215.929**, while **ETS Holt-Winters** places second with test sMAPE **27.217** and MAE **4816.454**.

The gap between the top two and the rest indicates that quarterly forecasting still rewards models that explicitly carry both trend and seasonal structure. Simpler baselines remain useful reference points, but they flatten the recent level shift too aggressively.

```

In [9]: # =====
# SECTION 8: COMPARISON TABLE
# =====
# Aggregate all model results, add interpretive notes, and rank by test performance

# Define interpretive notes for each model based on its expected behavior
notes_map = {
    "Naive": "Too biased",
    "Moving Average": "Too flat",
    "Seasonal Naive": "Misses trend/regime shifts",
    "ETS SES": "May underfit trend",
    "ETS Holt-Winters": "Can over/under-smooth peaks",
    "ARIMA(1,1,1)": "No explicit seasonality",
    "SARIMA(1,1,0)(1,1,0,4)": "Strong seasonal baseline",
    "Theta": "Strong trend-seasonality balance",
    "FFT": "Seasonal frequency focus"
}

# Convert eval_rows list to DataFrame and attach notes
results = pd.DataFrame(eval_rows)
results["Notes"] = results["Model"].map(notes_map).fillna("")

# Sort by test performance (primary: sMAPE_test, secondary: MAE_test, tertiary: run
results = results.sort_values(["sMAPE_test", "MAE_test", "runtime_sec"]).reset_index()

# Select columns for display and round metrics to 3 decimal places
comparison_cols = ["Model", "sMAPE_val", "MAE_val", "RMSE_val", "sMAPE_test", "MAE_
results_display = results[comparison_cols].copy()
for c in ["sMAPE_val", "MAE_val", "RMSE_val", "sMAPE_test", "MAE_test", "RMSE_test"
    results_display[c] = results_display[c].round(3)

# Display the comparison table
display(results_display)

```

	Model	sMAPE_val	MAE_val	RMSE_val	sMAPE_test	MAE_test	RMSE_test	runtime
0	SARIMA(1,1,0) (1,1,0,4)	39.487	4722.176	5166.195	17.814	3215.929	3493.053	(
1	ETS Holt-Winters	18.631	1817.996	2062.607	27.217	4816.454	5388.450	(
2	ARIMA(1,1,1)	15.809	1503.605	1701.869	31.418	5448.814	6117.441	(
3	Theta	12.753	1201.793	1409.581	36.802	6213.312	6989.088	(
4	Naive	13.575	1280.636	1544.024	37.826	6362.273	7202.638	(
5	ETS SES	13.559	1279.082	1543.301	39.401	6573.574	7416.138	(
6	FFT	16.280	1490.151	1979.799	51.075	8048.722	8848.626	(
7	Moving Average	20.351	1815.182	2339.762	54.138	8408.727	9082.617	(
8	Seasonal Naive	28.723	2444.091	2846.389	56.317	8597.545	9289.224	(

9. Forecastability Assessment

The quarterly credit card series is also **moderately forecastable**, with somewhat smoother residual structure after aggregation than the monthly version. The current diagnostics show:

- **CoV = 0.6821**: overall variability remains substantial relative to the mean level
- **Residual Variability = 0.0563**: quarterly aggregation removes a meaningful amount of short-run noise, leaving a cleaner underlying signal
- **Spectral Entropy = 0.5042**: the series still contains exploitable structure, but it is less regular than a near-deterministic seasonal process

This profile supports using explicit seasonal baselines for the quarterly benchmark. It also helps explain why the leading SARIMA model separates cleanly from the flatter alternatives on the test set.

```
In [10]: # =====
# SECTION 9: FORECASTABILITY ASSESSMENT
```

```

# =====
# Compute objective metrics to assess how predictable the series is

# ---- Coefficient of Variation (CoV) ----
# Ratio of standard deviation to mean; <0.5 = easy, >0.8 = hard to forecast
cov = float(y.std(ddof=1) / y.mean())
if cov < 0.5:
    cov_label = "easy"
elif cov > 0.8:
    cov_label = "hard"
else:
    cov_label = "moderate"

# ---- Residual Variability (RV) ----
# Variability remaining after removing trend and seasonality via decomposition
if len(y) >= 2 * 4:
    decomp = seasonal_decompose(y.to_timestamp(how="end"), model="additive", period
    resid = pd.Series(decomp.resid).dropna()
    rv = float(resid.std(ddof=1) / y.mean()) if len(resid) else np.nan
else:
    rv = np.nan

# ---- Spectral Entropy ----
# Normalized entropy of power spectral density (0-1 scale)
# Higher entropy = more uniform frequency distribution = less predictable dynamics
spectral_entropy = compute_spectral_entropy(y.values)

# Compile forecastability metrics into interpretable DataFrame
forecastability_df = pd.DataFrame([
    {"Metric": "CoV", "Value": cov, "Interpretation": f"{cov_label} ({'<0.5 easy, >
    {"Metric": "Residual Variability", "Value": rv, "Interpretation": "Lower is bet
    {"Metric": "Spectral Entropy", "Value": spectral_entropy, "Interpretation": "Hi
])

# Display the forecastability assessment
display(forecastability_df.round(4))

```

	Metric	Value	Interpretation
0	CoV	0.6821	moderate (<0.5 easy, >0.8 hard)
1	Residual Variability	0.0563	Lower is better after trend/seasonality removal
2	Spectral Entropy	0.5042	Higher implies less predictable dynamics

10. Best Baseline Plot + Selection

The current quarterly run selects **SARIMA(1,1,0)(1,1,0,4)** and **ETS Holt-Winters** as the two strongest interpretable baselines.

- **SARIMA(1,1,0)(1,1,0,4)** is the clear accuracy winner on the held-out quarterly test window

- **ETS Holt-Winters** remains the most competitive non-ARIMA alternative and provides a second structured benchmark
- The 2026 quarterly forecasts stay high in both models: SARIMA projects about **24.2k-27.9k** complaints per quarter, while Holt-Winters sits around **24.6k-26.9k**

Residual diagnostics are still part of the selection step so the chosen baselines are supported by both error metrics and basic fit-quality checks.

```
In [11]: # =====
# SECTION 10: BEST BASELINE PLOT & SELECTION
# =====
# Extract top 2 models (by test performance), run diagnostics, and produce forward

# ---- Residual Diagnostics for Top 2 Models ----
# Check residual mean/std and Ljung-Box autocorrelation test (Lag 4)
diag_df = run_residual_diagnostics(results, model_registry, top_n=2)
print("Residual Diagnostics for Top 2 Models:")
display(diag_df.round(4))

# ---- Select Top 2 Models and Generate 4-Quarter Forward Forecast ----
selected = results.head(2)["Model"].tolist() # Get best 2 by SMAPE_test, MAE_test,
h = FORECAST_HORIZON
future_index = pd.period_range(start=y.index[-1] + 1, periods=h, freq=FREQ_ALIAS)

# Re-fit each selected model on full data and generate future forecast
forecast_tbl = pd.DataFrame(index=future_index.astype(str))
for m in selected:
    # Re-fit model on full series (train+val+test) for production forecast
    if m == "Naive":
        fc = fc_naive(y, h)
    elif m == "Moving Average":
        fc = fc_mavg(y, h, k=4)
    elif m == "Seasonal Naive":
        fc = fc_snaive(y, h, season=SEASONAL_PERIODS)
    elif m == "ETS SES":
        fc = SimpleExpSmoothing(y, initialization_method="estimated").fit().forecast
    elif m == "ETS Holt-Winters":
        fc = ExponentialSmoothing(y, trend="add", seasonal="add", seasonal_periods=
    elif m == "ARIMA(1,1,1)":
        fc = ARIMA(y, order=(1, 1, 1), trend="t").fit().forecast(h).values
    elif m == "SARIMA(1,1,0)(1,1,0,4)":
        fc = SARIMAX(y, order=(1, 1, 0), seasonal_order=(1, 1, 0, SEASONAL_PERIODS)
    elif m == "Theta":
        fc = ThetaModel(y, period=SEASONAL_PERIODS, deseasonalize=True).fit().forec
    elif m == "FFT":
        fc = fc_fft(y, h, keep=6)
    else:
        continue
    forecast_tbl[m] = np.round(fc, 1)

print("\n4-Quarter Forward Forecast from Top 2 Baselines:")
display(forecast_tbl)

# ---- Plot Observed Series + Top 2 Baseline Forecasts ----
```

```

fig, ax = plt.subplots(figsize=(10, 4))
y_ts = pd.Series(y.values, index=y.index.to_timestamp(how="end"))
ax.plot(y_ts.index, y_ts.values, label="Observed", color="steelblue")

for m in selected:
    idx = future_index.to_timestamp(how="end")
    ax.plot(idx, forecast_tbl[m].values, marker="o", label=f"Forecast - {m}")

ax.set_title("Top Baseline Forecasts (Next 4 Quarters)")
ax.legend()
plt.tight_layout()
plt.show()

# Print summary
print("\nSelected strong baselines:", selected)
print("Rationale: lowest test sMAPE/MAE with reasonable runtime, then checked diagn

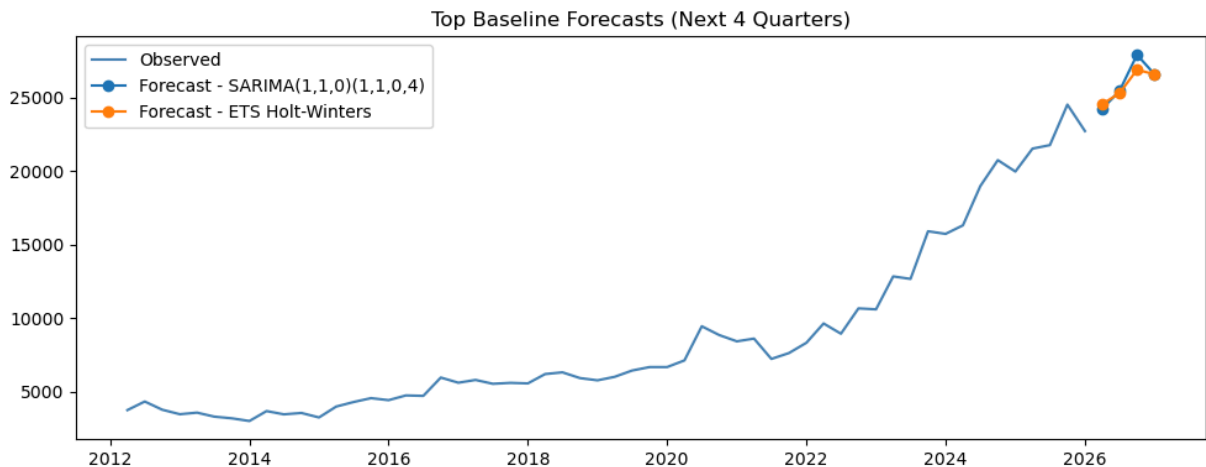
```

Residual Diagnostics for Top 2 Models:

	Model	resid_mean	resid_std	ljung_box_p_lag4
0	SARIMA(1,1,0)(1,1,0,4)	16.9722	1003.2928	0.3915
1	ETS Holt-Winters	117.7134	645.1730	0.8202

4-Quarter Forward Forecast from Top 2 Baselines:

	SARIMA(1,1,0)(1,1,0,4)	ETS Holt-Winters
2026Q1	24243.9	24565.1
2026Q2	25454.3	25304.4
2026Q3	27919.3	26900.0
2026Q4	26594.6	26597.4



Selected strong baselines: ['SARIMA(1,1,0)(1,1,0,4)', 'ETS Holt-Winters']

Rationale: lowest test sMAPE/MAE with reasonable runtime, then checked diagnostics for top models.

Auto Interpretation (Model-Specific)

```
In [12]: # =====
# SECTION 11: AUTO INTERPRETATION (MODEL-SPECIFIC)
# =====
# Generate automated summary of model selection: which models won and why others lost

# Identify selected (top 2) and non-selected models
selected_models = results.head(2)["Model"].tolist()
selected_set = set(selected_models)

# Get best model's performance for comparison
best_smape = float(results.iloc[0]["sMAPE_test"])
best_mae = float(results.iloc[0]["MAE_test"])

# ---- Print Selected Baselines ----
print("Strong baselines selected:")
for i, m in enumerate(selected_models, start=1):
    row = results[results["Model"] == m].iloc[0]
    print(f"{i}. {m} | sMAPE_test={row['sMAPE_test']:.2f}, MAE_test={row['MAE_test']:.2f}, runtime={row['runtime']:.4f}s")

# ---- Explain Why Other Models Lost ----
print("\nWhy other models lost:")
losers = results[~results["Model"].isin(selected_set)].copy()
for _, row in losers.iterrows():
    gap_smape = float(row["sMAPE_test"] - best_smape)
    gap_mae = float(row["MAE_test"] - best_mae)
    note = row.get("Notes", "")
    print(
        f"{row['Model']}: {note}; +{gap_smape:.2f} sMAPE and +{gap_mae:.1f} MAE vs best model."
    )

# ---- Explain Selection Rationale ----
print("\nSelection rationale:")
print("Lowest test sMAPE/MAE first, then runtime and residual diagnostics for the top candidates.")
```

Strong baselines selected:

1. SARIMA(1,1,0)(1,1,0,4) | sMAPE_test=17.81, MAE_test=3215.9, runtime=0.1172s
2. ETS Holt-Winters | sMAPE_test=27.22, MAE_test=4816.5, runtime=0.0816s

Why other models lost:

- ARIMA(1,1,1): No explicit seasonality; +13.60 sMAPE and +2232.9 MAE vs best model.
- Theta: Strong trend-seasonality balance; +18.99 sMAPE and +2997.4 MAE vs best model.
- Naive: Too biased; +20.01 sMAPE and +3146.3 MAE vs best model.
- ETS SES: May underfit trend; +21.59 sMAPE and +3357.6 MAE vs best model.
- FFT: Seasonal frequency focus; +33.26 sMAPE and +4832.8 MAE vs best model.
- Moving Average: Too flat; +36.32 sMAPE and +5192.8 MAE vs best model.
- Seasonal Naive: Misses trend/regime shifts; +38.50 sMAPE and +5381.6 MAE vs best model.

Selection rationale:

Lowest test sMAPE/MAE first, then runtime and residual diagnostics for the top candidates.

11. Conclusion

Using the trimmed quarterly credit card series from 2012-01-01 through 2025-12-31 , this notebook establishes a focused classical benchmark for CFPB credit card complaint forecasting.

Key Findings:

- **Best Quarterly Baseline: SARIMA(1,1,0)(1,1,0,4)** ranks first on the held-out test set with sMAPE **17.814** and MAE **3215.929**
- **Runner-Up: ETS Holt-Winters** ranks second with sMAPE **27.217** and MAE **4816.454**
- **Forecastability:** the quarterly series is **moderately forecastable** with CoV **0.6821**, Residual Variability **0.0563**, and Spectral Entropy **0.5042**
- **Behavioral Pattern:** quarterly aggregation smooths some short-run noise, but models still need explicit trend and seasonality to keep up with the later level shift

Implication: The quarterly benchmark to carry forward is seasonal SARIMA first, with Holt-Winters as the main non-ARIMA comparator. Their 2026 forecasts keep quarterly complaint volume elevated at roughly **24k-28k per quarter**, which provides the benchmark range for downstream models.